

Floating_ice_Sheet_for_exoplanets

Matlab model to study ice coverage of of water worlds

Alexandra Rochon

Intro

This model was adapted from Tziperman et al. 2012, originally aimed at studying Snowball Earth and the effects of continental configuration on ice flow. The model was modified to be able to use output from a GCM as input parameters, specifically a 2D temperature map and a 2D initial ice elevation map.

From this the model runs for a given amount of time, using ice flow equations to reach a steady state of ice coverage. It models floating ice shelf. There are no grounded ice equations in this model. Adding such equations causes numerical instabilities and has not been fully examined. It is unlikely that it will be possible to add grounded ice sheet equations to this model. A different model should be used to answer questions related to grounded ice. This model can have continent coverage, but they are by default ice free. The subglacial ocean is infinite by definition.

This model is therefore best adapted to represent a waterworld with global ocean coverage and a high water mass content. The current temperature map is adapted for a tidally-locked planet, with a hotspot at the substellar point and coolest regions at the pole. There are two other profiles for a fully rotating planet, a “cold” and a “warm” profile that are included in the model. A new profile can be added if needed by modifying `TS=T_surface_function(par)`.

The current simulations are run for an Earth-like planet, with Proxima Centauri b taken as a hypothetical case study.

Different continental configurations can be tested, by choosing a static mask (`par.which_mask = 1;`) and either using one of the preconfigured ones or creating one using `make_continents.m`. The preconfigured ones are from Tziperman et al. and include present Earth continental configuration, 630 ma Earth continental configuration and examples of idealized channels to study the effects of continental constriction on ice velocity.

The model defines the substellar ocean as an ice-free mask that can be modified at each iteration, whereas the continental mask is static. The ocean mask is defined from the initial ice elevation. Both a static mask and a dynamic mask can be used at the same time if needed.

The model has a 2D grid of 176 by 176 defined in longitude and latitude. By convention, in the code, *i* is defined as longitude and *j* as latitude. The velocity vectors, *u* and *v*, are positive towards the east and towards the equator respectively.

When the model runs, it prints each completed incrementation (*n*), and issues if they arise (due to numerical instability, NaN or 0 where there shouldn't be, or negative *h* (ice elevation), suggesting ice is flowing too fast or the timesteps are too big). The total duration of the model run (in "steps") and the time representation by each iteration/step, is also printed. If the viscosity doesn't converge, this will also be printed (20 iterations max to converge). Keep in mind that it can take multiple days for the model to finish running. The model will not automatically stop running if there is an error, and there are traps to try to correct the errors.

Limitations

- There is no feedback between the ice elevation and the surface temperature. Initial ice elevation maxima are around 3 km with the current simulation data. There would be an effect on the temperature when the elevation is reduced to 1km or less, but this isn't included in the model right now.
- The surface temperature map and the initial ice elevation is provided by the GCM. A GCM could be fed the outputs of the model (final ice elevation) to recalculate surface temperature and precipitation / accumulation rates. However, because this is a floating ice model, the timescale at which the model reaches steady state (or approaches it) is much much faster than the timescale of the GCM. For this reason, it is complex to couple them in a self-consistent way.
- The model is highly susceptible to numerical instabilities. Big timesteps and high ice velocities cause the ice elevation to reach negative levels. The calculations will result in NaN.

Limitations (cont.)

- Continents are overlaid on top of initial ice elevation map, so the continental map effectively deletes the ice where continents are created. So big continental coverage means very little ice mass.
- There is no grounded ice in the model, only floated ice. Continents are, by definition, ice-free.
- When ice reaches the substellar ocean, or areas of surface temperatures above the freezing point of water, the surface temperature is manually decreased to below the point of freezing (265K) to allow the ice to progress. This is likely what would happen if sea ice dynamics were taken into account. Otherwise, every pixel which receives starlight to heat up above 273K could never be covered in ice. This is done at line 1081 and could be adapted with a more rigorous approach.

Limitations (cont.)

- Calculations of Glen's flow law are the most time-intensive part of the whole code. It is calculated at every incrementation of the vertical temperature profile for each pixel. The vertical temperature profile is defined as $T = T_{\text{surface}}(i, j) : 0.5 : \text{par}.T_f$, where $\text{par}.T_f$ is the freezing point of water. Taylor expanding the equation could maybe approximate the numerical answer, but it hasn't been done. To improve efficiency, the number of incrementation of the vertical temperature profile was reduced. There is a test already included in the code that was used to ensure that precision is within 5% of the initial incrementation value. For the ice coverage to completely even out, the timescale is of the order of ~10k years to ~100k years, which takes a very long time for the model to run even on a cluster like Compute Canada. It can take upwards of 7 days which is the maximum computing power it has been tested with currently. At this point, the substellar ocean is completely closed and the ice elevation is almost completely even. There is an exponential decrease of the maximum ice elevation point, so reaching a completely homogeneous ice elevation takes much longer than losing the substellar ocean and reducing the pole ice caps.

Adaptations

- To study the effects of a non-synchronous rotation, the temperature profile needs to be changed. At every time step, the profile is recalculated by calling the function `TS=T_surface_function(par)`. A calculation of the rotation can therefore be added to this function. By changing the if statement, a plot of the temperature profile will automatically be created at each time step (currently only at the beginning)
- It is possible to calculate a freezing / melting forcing of the ice. This is not currently implemented because it didn't match up with the use of the GCM coupling. However, residual code from the original model is still included and can be uncommented to be used. The function `S=S_total(par)` can be used to calculate surface ice accumulation from a 1D climate model. The function `S=S_bot(par,h)` is used to calculate a basal freeze / melt rate. The output of both of these questions is currently set to 0.

Adaptations (contd.)

- The ice elevation (and other parameters) are saved every x iteration. By knowing the total number of timesteps and the time (in years) between each (usually 0.5 or 0.1 years), we can reconstruct the evolution of a parameter as a function of time. However, the exact number of times something is saved can change if there is an issue and it might not be the most accurate way to extract time from the results. Implementing a way to save the exact time into the netCDF output file would be useful to ensure accurate timing.

The Files

In the local directory, there are multiple files required for the model to run:

Floating_ice_sheets.m This is the main code for the model. This executes the input from the experiment files.

exp_00.m The input parameters of each experiments are described here. Multiple exp_*.m files can be defined for array jobs. Here we specify the type of temperature profile (“mine” to provide one as input), the path to the input ice elevation map produced by the GCM, the timescale the experiment should run for and the incrementations, and which mask is used. This is found in the input folder.

The Files (contd.)

GCM_input.nc A NetCDF file containing the initial ice elevation and surface temperature. This is normally produced by the GCM. The path to this file needs to be given in the exp_00.m file. The values in this file are interpolated to the grid definition used by the model, which is 176 by 176, in longitude and latitude.

parameters.mat A matrix containing the par structure which contains all of the planetary parameters and constants needed for the code. This is found under the input folder.

set_parameters.m The code to set up the par structure and save it under parameters.mat. The planetary parameters can be modified here to describe a different planet. Currently “Earth-like” parameters are used, but this would appropriately represent a planet like Proxima b. Some values are rounded. Any value can also be overwritten in the main code once parameters.mat is imported (line 27). All parameters are defined such that if something is omitted from the exp_00.m file, the code will still run.

The Files (contd.)

make_continents.m is used to create continental configurations. These are just 176 by 176 binary maps, where 0 are ice free pixels. The output matrix is saved under input/mask/ as continent.mat and the name can be changed if multiple examples of continents are created.

Restart This folder contains matrices of the final parameters after a model run. This matrix is updated at every x timestep as defined in parameters.mat, usually the same frequency as the plotting frequency. Any experiment can therefore be restarted with the final parameters of h (ice elevation), u_n , v_n (velocity vectors), mask, and B (viscosity). If this is used, this will overwrite the input GCM ice elevation with the h value of ice elevation. The code will automatically look for a restart file with the same experiment code.

The Files (contd.)

output_analysis.m This file contains codes to make a few plots made to analyze the output of the model. It uses the NetCDF output file, not the one made to couple with the GCM, as it contains a few more fields. Specifically, it has the ice elevation saved at every n iteration to be able to construct an ice evolution plot for example. The GCM output file only contains the final ice elevation value.

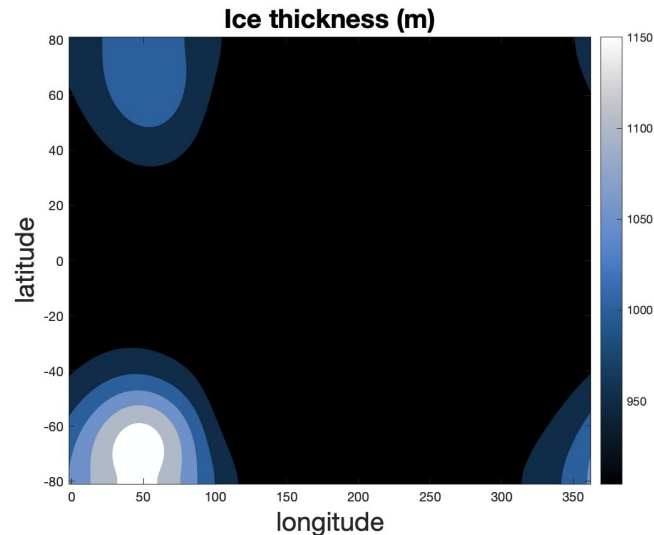
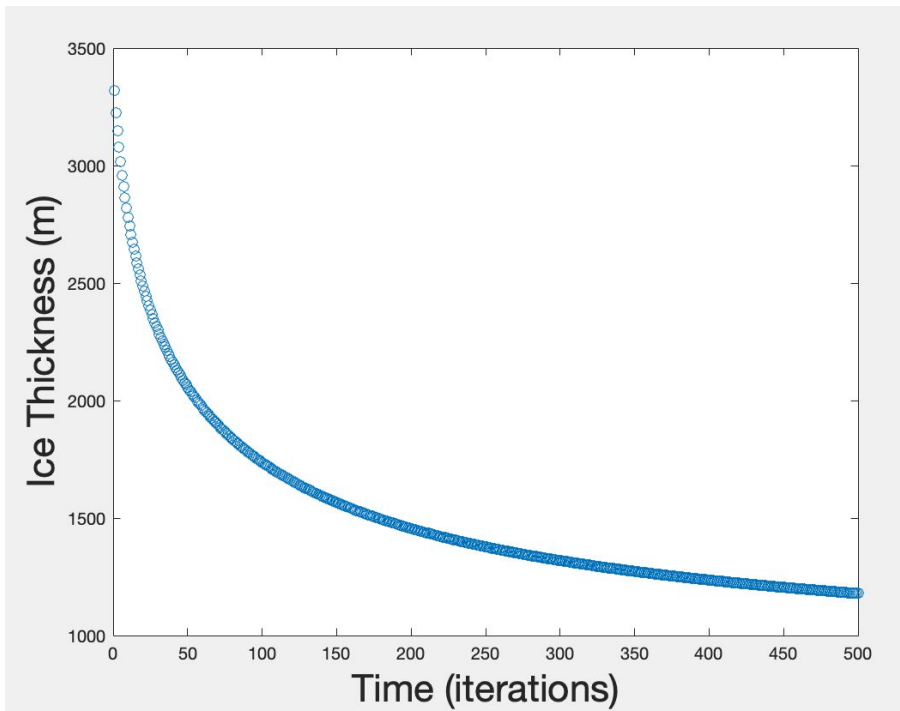
Notes

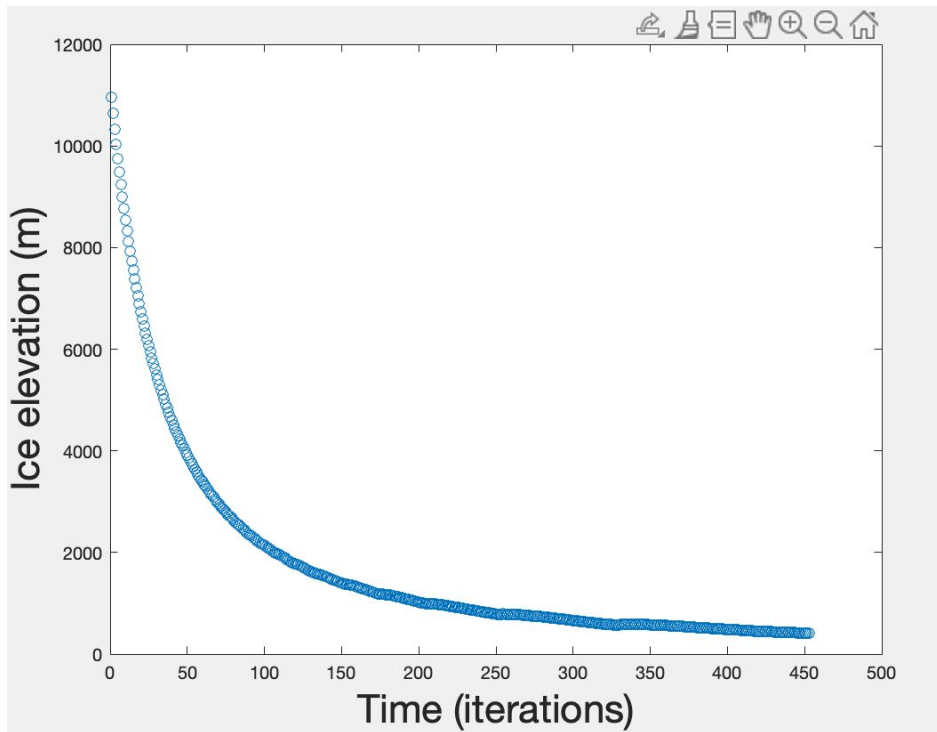
- The path/to/files should be checked before running the code for the first time. Some places have the full path hard-coded which will cause issues.
- The size of each time step is < 1 year in order to avoid numerical instabilities. It is usually half a year, 0.1 year or 0.01 year. This is why the timescale with the GCM is hard to couple and why the model takes so long to run.
- The length of the timesteps isn't saved in the output matrix and neither is the saving frequency (which correlates to the total number of integrations). These two values are necessary to convert the results in time unit. They are printed by the code at the beginning and are in the exp_00.m file, so a different experiment number should be used every time.

The figures

Some figures are generated as the model runs at every n iterations. Some extra figures are generated at either the first or last time step and the `output_analysis.m` file has code to make extra figures with the results.

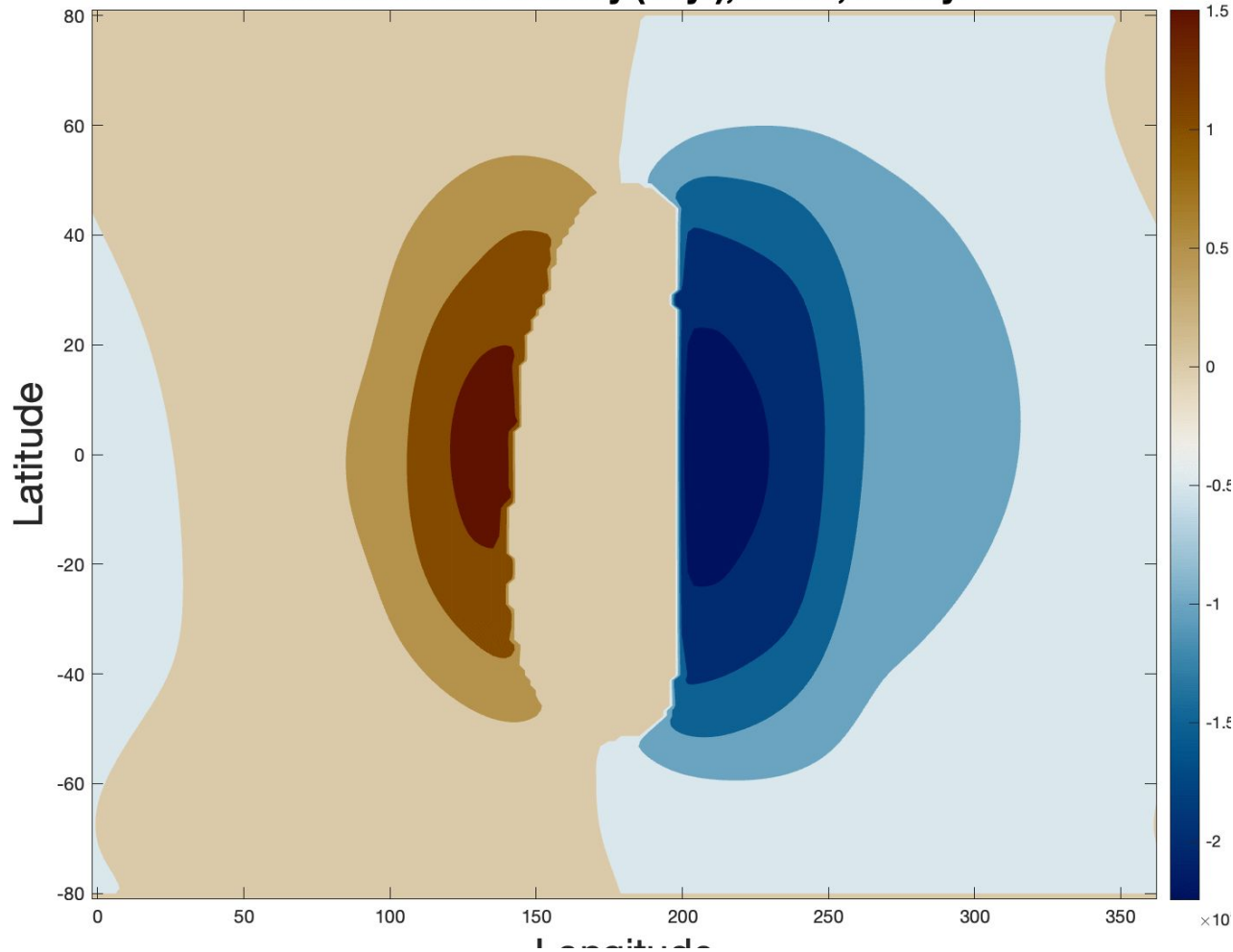
This is an example of how the maximum ice elevation evolves with time (left). These iterations don't necessarily correspond to every iteration ran by the model, only to the frequency of saving (usually every 250 iterations, specified in exp_0.m). This model run lasted 7 days on Compute Canada and represents 10k years. There is 250 m between the lowest point and the highest point (pole) after this time period.





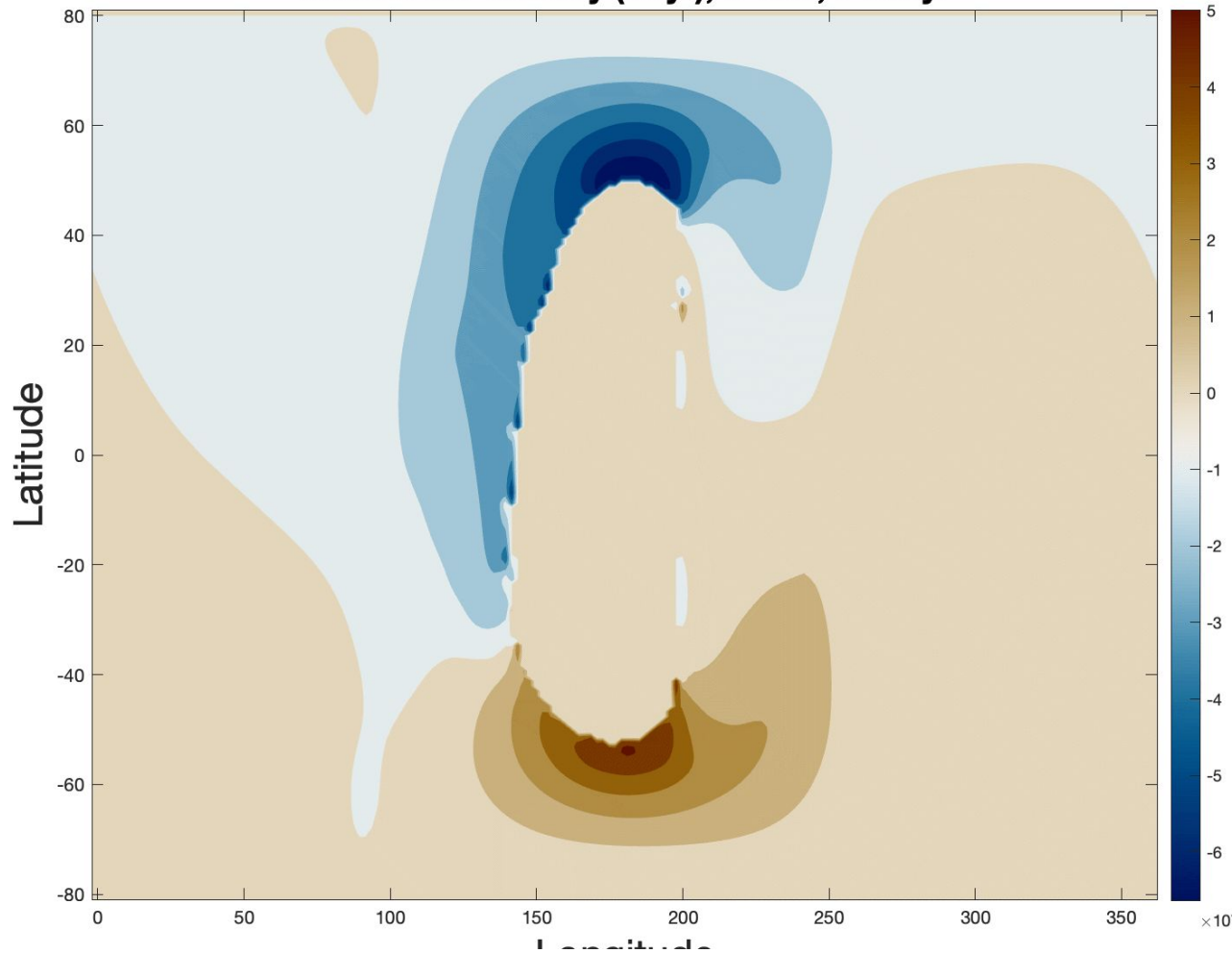
This plot is a “first derivative” plot. At each timestep, the difference in ice elevation compared to the previous iteration is calculated at each pixel and summed. This value is plotted on the Y axis for every iteration (X axis). This plot can be made with the file `output_analysis.m`

Horizontal Ice Velocity (m/yr), $n=500$, $t=50\text{kyr}$

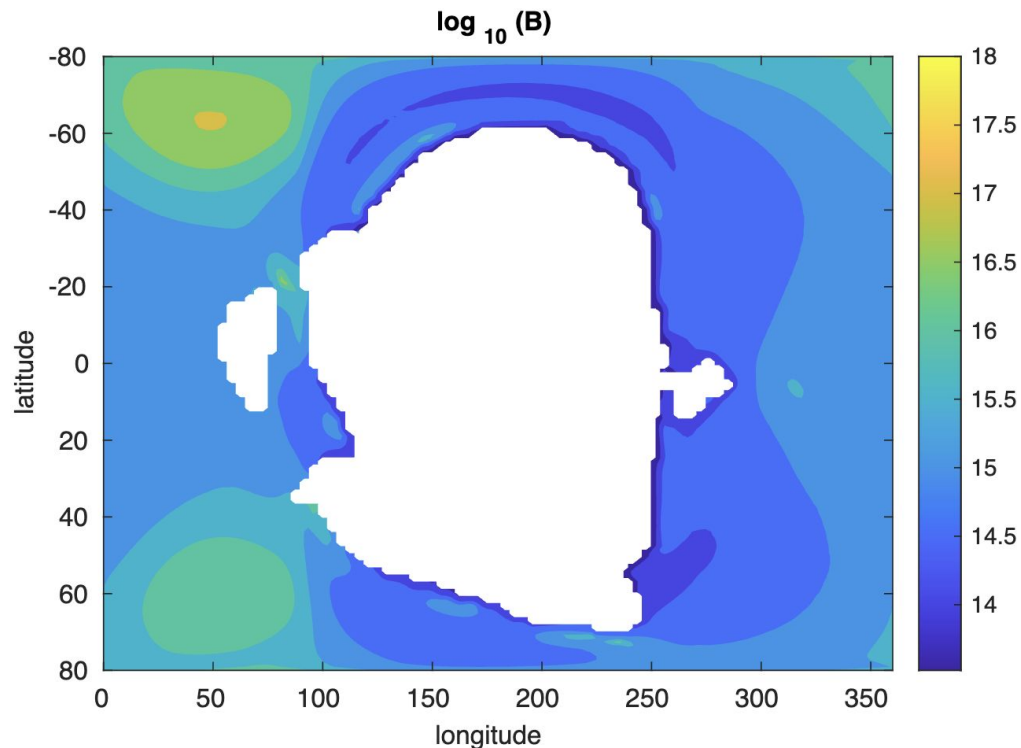


This is an example of the evolution of the longitudinal velocity as a function of time. It decreases significantly as time goes on.

Vertical Ice Velocity (m/yr), n=500, t~50kyr



This is an example of the evolution of the latitudinal velocity as a function of time. It decreases significantly as time goes on.

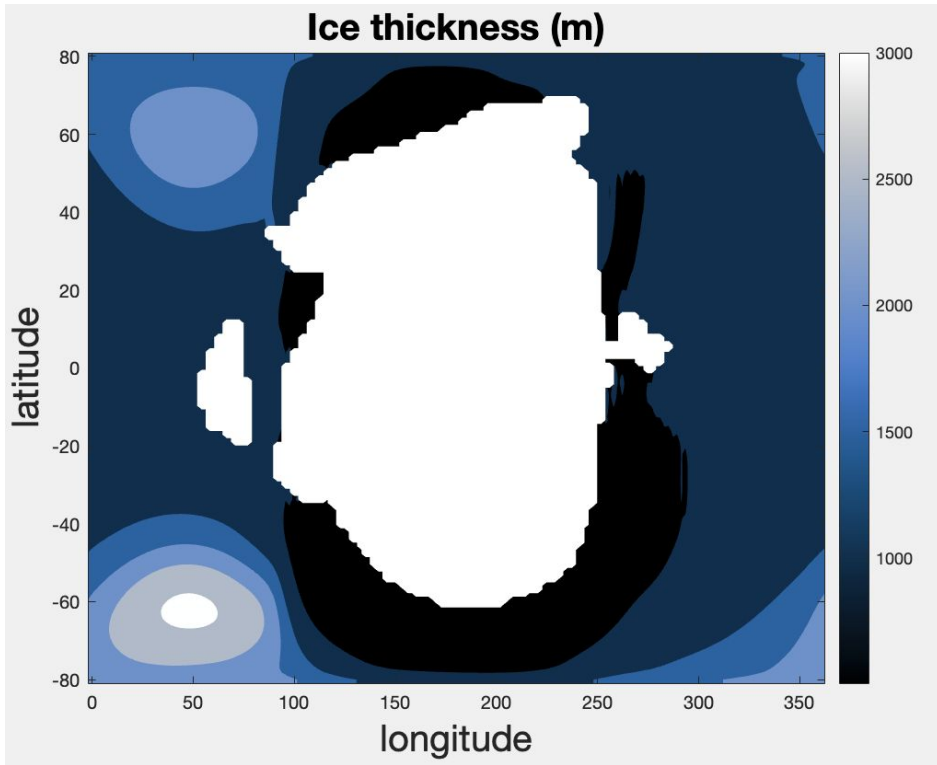


A figure of the log of the viscosity at each gridpoint is produced at each plotting interval. Areas of higher viscosity correspond to higher ice mass and more movement.

Some test cases

- Continents delete ice mass, because the continents are define as ice-free. This accelerates the process of evening out the ice elevation is there is a big continent at either pole, where most of the ice mass is concentrated. For a small continent located near the substellar ocean, the effects are minimal.

MAIN RESULT: Since this is a floating ice model, the open ocean at the substellar point becomes covered in ice extremely rapidly on the order of $\sim$ a few years. Hence, a waterworld would have a surface ocean only if the insolation was sufficient to melt the ice at the substellar point. (see gif)



This is an example of an idealized continent map, laid over the initial ice elevation map. In black is the open ocean which is slowly diminishing whereas the white caps at the poles show highest ice elevation area. This example wasn't run until steady state, but the continental coverage produces similar result, just on a shorter time scale since total ice coverage is reduced. The specific impact of continents hasn't been assessed in more details.